

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

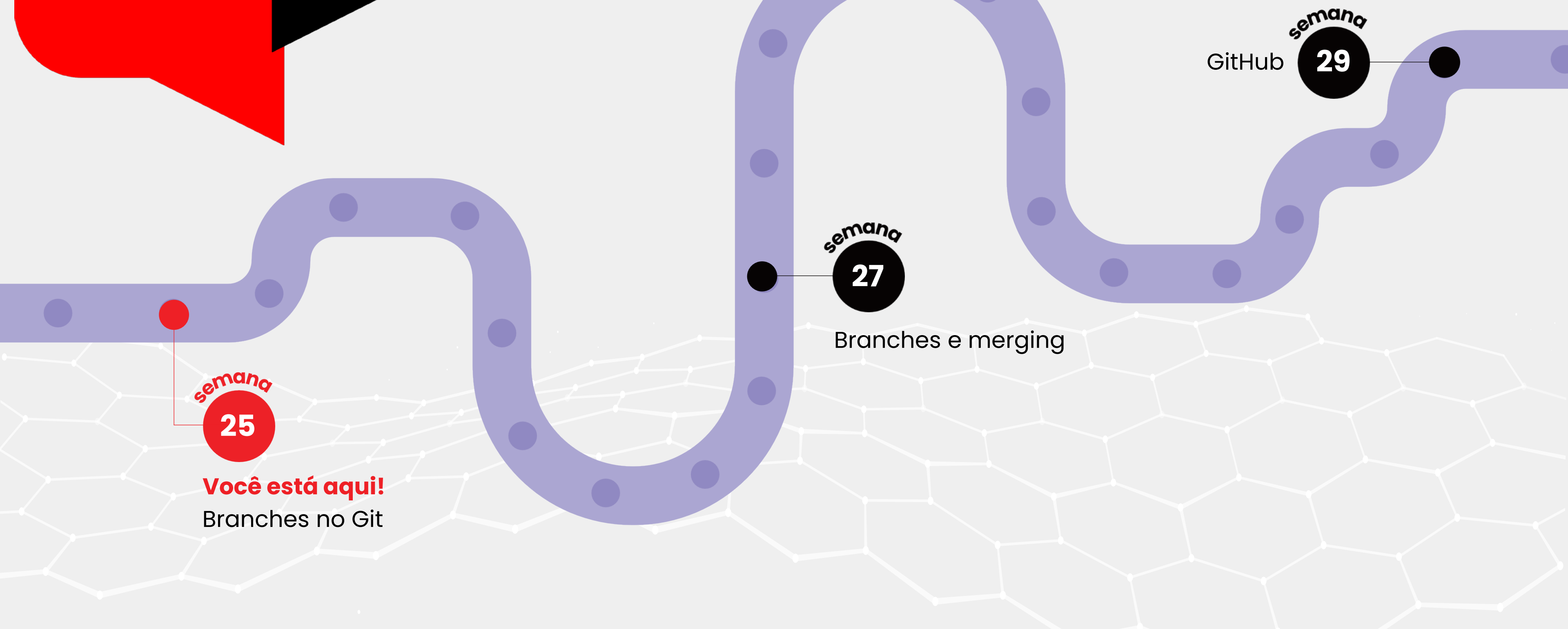
Branches no Git

Operações avançadas com *branches* no Git

Código da aula: [DADOS]ANO1C1B4S25A2

Controle de versão
com Git e GitHub

Mapa da Unidade 8 Componente 1



Controle de versão
com Git e GitHub

Mapa da
Unidade 8
Componente 1

Você está aqui!

Branches no Git

**Operações avançadas com
branches no Git**

Código da aula: [DADOS]ANO1C1B4S25A2

25



Objetivos da aula

- Introduzir e utilizar ferramentas de desenvolvimento de software como Git e GitHub.



Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou internet.



Duração da aula

50 minutos.



Competências técnicas

- Usar ferramentas de desenvolvimento de software como Git e GitHub;
- Compreender e dominar técnicas de manipulação de dados.



Competências socioemocionais

- Analisar informações de forma crítica e tomar decisões;
- Avaliar a qualidade dos dados e a eficácia dos modelos e das técnicas de análise;
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais, colaborando com colegas, gestores e clientes.

Construindo
o **conceito**

Branching e merging avançados

Branching e merging avançados incluem técnicas e comandos adicionais que permitem um controle mais refinado sobre como branches são criados, gerenciados e integrados.

Isso pode incluir o uso de branches temporários, merges com estratégias específicas e ferramentas adicionais para facilitar o processo.



Tome nota

Branching e merging avançados são essenciais para gerenciar projetos grandes e complexos. Além dos comandos básicos, existem técnicas adicionais para criar e gerenciar branches de forma mais eficiente. Por exemplo, branches temporários podem ser usados para testar novas funcionalidades ou corrigir bugs sem afetar o branch principal.

Construindo
o **conceito**

Rebase x merge

Rebase e **merge** são duas estratégias diferentes para integrar mudanças de diferentes branches.

O **rebase** move ou aplica commits de um branch para outro, criando um histórico de commit linear.

O **merge** combina as mudanças de diferentes branches em um único commit, preservando o histórico de commits original.

Construindo
o **conceito**

Cherry-picking

Cherry-picking é a prática de aplicar um commit específico de um branch para outro, permitindo que mudanças individuais sejam integradas sem precisar fazer um merge completo de branches.



Tome nota

Isso é útil quando é preciso integrar mudanças específicas sem afetar outras partes do branch. O comando **git cherry-pick <commit>** é usado para isso.

Construindo o conceito

Branching workflows

Branching workflows referem-se a diferentes metodologias e práticas de gerenciamento de branches em um projeto. Alguns exemplos incluem: **Git Flow**, **GitHub Flow** e **GitLab Flow**, cada um com suas próprias regras e estratégias para criação, integração e lançamento de branches.

Git Flow	GitHub Flow	GitLab Flow
É um modelo de branching que define como branches são criados, gerenciados e integrados, com diferentes branches para desenvolvimento, releases e hotfixes.	É uma abordagem mais simples, com foco em um único branch de desenvolvimento e branches de features.	Combina elementos de ambos, com suporte para múltiplos ambientes e integração contínua.

Elaborado especialmente para o curso.

Construindo
o **conceito**

Resolução de conflitos avançada

A resolução de conflitos avançada envolve o uso de ferramentas e técnicas adicionais para resolver conflitos de merge de maneira eficiente.



Tome nota

Isso pode incluir o uso de ferramentas gráficas, scripts de automação e técnicas para identificar e resolver conflitos complexos.

Construindo
o **conceito**

Exemplos práticos

Configurando o Git no Jupyter Notebook

1. Configurar o nome de usuário e o e-mail do Git:

```
# Configura o nome de usuário do Git
!git config --global user.name "Seu Nome"

# Configura o e-mail do usuário do Git
!git config --global user.email "seu_email@example.com"
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplos práticos

2. Refazer os exemplos simples: Exemplo 1 – Criar e mudar para um novo branch:

```
import os

# Cria uma pasta de trabalho para o repositório
os.makedirs("my_git_project", exist_ok=True)
%cd my_git_project

# Inicializa o repositório Git
!git init

# Configura o nome e o e-mail do Git
!git config user.name "Seu Nome"
!git config user.email "seu_email@example.com"

# Cria o arquivo sum.py e faz o commit inicial
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

print(add_numbers(2, 3))""")

!git add sum.py
!git commit -m "Initial commit"

# Cria um novo branch chamado 'feature1' e muda para ele
!git checkout -b feature1
```

```
# Adiciona uma nova função no branch 'feature1'
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

def multiply_numbers(a, b):
    return a * b # Adicionado no feature1

print(add_numbers(2, 3))
print(multiply_numbers(3, 4))""")

!git add sum.py
!git commit -m "Added multiply_numbers function in feature1"

# Volta para o branch 'main'
!git checkout main
```

Elaborado especialmente
para o curso com a
ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplos práticos

2. Refazer os exemplos simples: Exemplo 2: Merge simples:

```
# No branch 'main', adiciona outra função
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

def subtract_numbers(a, b):
    return a - b # Adicionado no main

print(add_numbers(2, 3))
print(subtract_numbers(5, 2))""")

!git add sum.py
!git commit -m "Added subtract_numbers function in main"

# Faz o merge do branch 'feature1' no 'main'
!git merge feature1

# Mostra o conteúdo final do arquivo após o merge
with open("sum.py", "r") as file:
    print(file.read())
```

Elaborado especialmente
para o curso com a
ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplos práticos

2. Refazer os exemplos simples: Exemplo 3: Rebase simples:

```
# Reset o branch 'feature1' para recriar o exemplo de rebase
!git checkout feature1
!git reset --hard HEAD~1

# No branch 'feature1', adiciona a função novamente
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

def multiply_numbers(a, b):
    return a * b # Adicionado no feature1

print(add_numbers(2, 3))
print(multiply_numbers(3, 4))""")

!git add sum.py
!git commit -m "Added multiply_numbers function in feature1"
```

Elaborado especialmente para o curso
com a ferramenta Jupyter Notebook.

```
# Muda para o branch 'main' e adiciona outra função
!git checkout main
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

def subtract_numbers(a, b):
    return a - b # Adicionado no main

print(add_numbers(2, 3))
print(subtract_numbers(5, 2))""")

!git add sum.py
!git commit -m "Added subtract_numbers function in main"

# Faz o rebase do branch 'feature1' no 'main'
!git checkout feature1
!git rebase main

# Volta para o branch 'main' e faz o merge
!git checkout main
!git merge feature1

# Mostra o conteúdo final do arquivo após o rebase
with open("sum.py", "r") as file:
    print(file.read())
```




Colocando
em **prática**

Exercício: Cherry-picking



20 minutos



Em duplas



Documento texto

1 Organizem-se em duplas.

2 Analisem os códigos apresentados no próximo slide e respondam: o que esse código está fazendo?
Descrevam o passo a passo.

3 Enviem o documento final para o AVA
(Ambiente Virtual de Aprendizagem).

Colocando
em **prática**

Exercício: Cherry-picking

```
import os

# Cria uma pasta de trabalho para o repositório
os.makedirs("my_git_project", exist_ok=True)
%cd my_git_project

# Inicializa o repositório Git
!git init

# Configura o nome e o e-mail do Git
!git config user.name "Seu Nome"
!git config user.email "seu_email@example.com"

# Cria o arquivo sum.py e faz o commit inicial
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

print(add_numbers(2, 3))""")

!git add sum.py
!git commit -m "Initial commit"
```

```
# Cria o branch feature2 e adiciona uma nova função
!git checkout -b feature2
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

def divide_numbers(a, b):
    if b != 0:
        return a / b
    else:
        return "Division by zero error" # Adicionado no feature2

print(add_numbers(2, 3))
print(divide_numbers(6, 0))""")

!git add sum.py
!git commit -m "Added divide_numbers function with zero check in feature2"

# Obtém o hash do commit
commit_hash = !git log -1 --pretty=format:%H

# Volta para o branch main e faz o cherry-pick do commit específico
!git checkout main
!git cherry-pick {commit_hash[0]}

# Mostra o conteúdo final do arquivo após o cherry-pick
with open("sum.py", "r") as file:
    print(file.read())
```



20 minutos



Em duplas



Documento texto

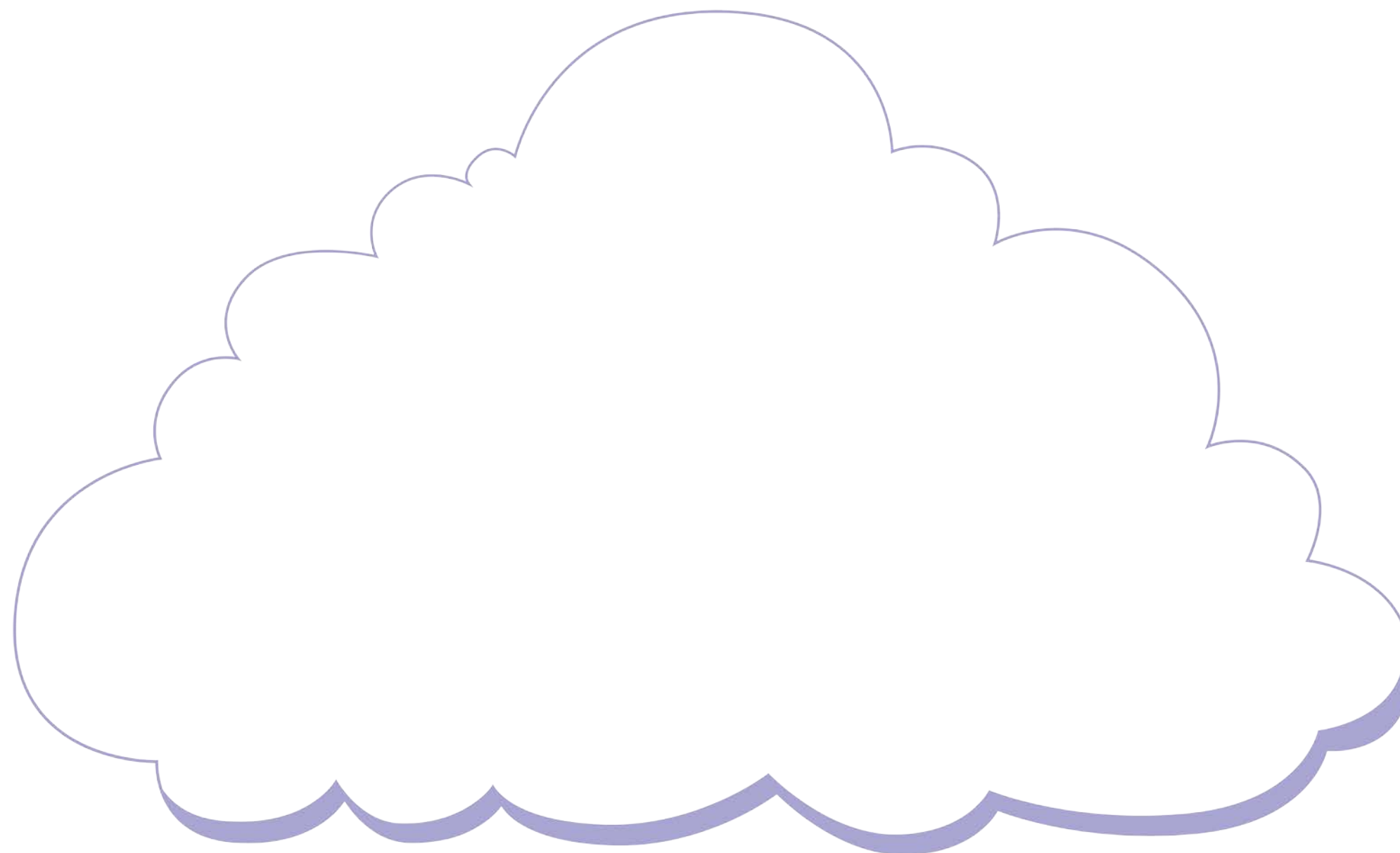
Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Nuvem de palavras



© Getty Images

O que nós
**aprendemos
hoje?**





© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** Branching e merging avançados são técnicas essenciais para gerenciar projetos grandes e complexos, permitindo a criação e a integração eficientes de branches sem comprometer o branch principal.
- 2** Rebase e merge são duas abordagens diferentes para integrar mudanças de diferentes branches. Rebase cria um histórico de commits linear, enquanto merge preserva o histórico original.
- 3** Cherry-picking permite aplicar commits específicos de um branch para outro, facilitando a integração de mudanças individuais sem a necessidade de um merge completo.

Saiba mais

Quer dominar o Git e GitHub para levar seus projetos para o próximo nível? Nesse curso completo, você aprenderá a compartilhar seu código, colaborar em projetos, gerenciar versões e muito mais!

ALURA. *Curso de Git e GitHub*: compartilhando e colaborando em projetos.

Disponível em: <https://cursos.alura.com.br/course/git-github-compartilhando-colaborando-projetos>.

Acesso em: 12 jul. 2024.

Referências da aula

BELL, P.; BEER, B. *Introdução ao GitHub*: um guia que não é técnico. São Paulo: Novatec, 2014.

GITHUB. *Documentação de introdução ao GitHub*, [s.d.]. Disponível em: <https://docs.github.com/pt/get-started>. Acesso em: 12 jul. 2024.

Identidade visual: imagens © Getty Images.

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**